Week #2: Computer Arithmetic and Error Analysis

Numerical Methods: Mathematical Foundations

June 7, 2025

Overview

Computer arithmetic forms the foundation of all numerical computations, yet its finite precision introduces subtle complexities that can dramatically affect algorithm behavior. This week we explore the mathematical principles of floating-point representation and analyze how roundoff errors arise, propagate, and can be mitigated through careful algorithm design.

The gap between continuous mathematics and discrete computation creates fundamental challenges that cannot be eliminated, only managed. We develop rigorous error bounds that quantify the limitations of finite-precision arithmetic and provide theoretical guarantees for numerical algorithms. These insights guide the development of numerically stable methods that maintain accuracy despite the inherent limitations of computer arithmetic.

Our approach emphasizes both theoretical understanding and practical implications, connecting abstract mathematical concepts to concrete computational considerations that affect all numerical methods.

---

1 Floating-Point Number Systems

1.1 Mathematical Representation

Definition. Floating-Point Number System

A floating-point number system $F(\beta, p, L, U)$ is characterized by:

- Base β (typically 2)
- Precision p (number of significant digits)
- Exponent range $[L, U]$

Numbers in this system have the form:

$$\pm d_0.d_1d_2\ldots d_{p-1} \times \beta^e$$

where $d_0 \neq 0$ (normalized form), $0 \leq d_i < \beta$, and $L \leq e \leq U$.

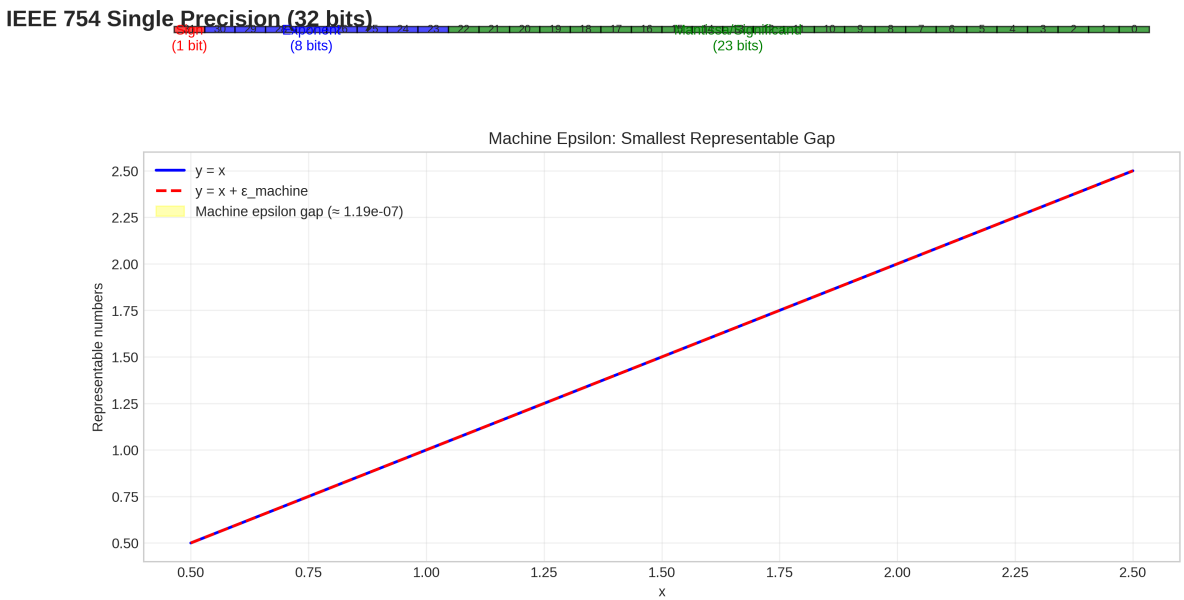

Figure 1: IEEE 754 floating-point representation. The top panel shows the bit layout for single precision (32-bit) format with sign, exponent, and mantissa fields. The bottom panel illustrates machine epsilon as the smallest representable gap between consecutive floating-point numbers.

Theorem 1.1 (Properties of Floating-Point Systems)**.** *For a floating-point system $F(\beta, p, L, U)$:*

1. *The smallest positive normalized number is β^L*
2. *The largest finite number is $(\beta - \beta^{-p+1}) \times \beta^U$*
3. *The spacing between consecutive numbers near x is approximately $\beta^{1-p}|x|$*
4. *The total number of representable finite numbers is $2(\beta - 1)\beta^{p-1}(\beta^{U-L+1} - 1) + 1$*

1.2 IEEE 754 Standard

Definition. IEEE 754 Standard

The IEEE 754 standard defines:

- **Single precision (32-bit):** 1 sign bit, 8 exponent bits, 23 mantissa bits
- **Double precision (64-bit):** 1 sign bit, 11 exponent bits, 52 mantissa bits
- **Special values:** ± 0, $\pm\infty$, NaN (Not a Number)
- **Rounding modes:** Round to nearest (default), round toward 0, round toward $+\infty$, round toward $-\infty$

Example 1.1 (IEEE 754 Representation)**.** *For single precision (32-bit):*

- *Exponent bias: 127*
- *Exponent range:* $[-126, 127]$
- *Machine epsilon:* $\varepsilon_{mach} = 2^{-23} \approx 1.19 \times 10^{-7}$

- *Smallest positive normalized number:* $2^{-126} \approx 1.18 \times 10^{-38}$
- *Largest finite number:* $(2 - 2^{-23}) \times 2^{127} \approx 3.40 \times 10^{38}$

Theorem 1.2 (IEEE 754 Guarantees)**.** *The IEEE 754 standard guarantees:*

1. *If x is a real number in the representable range, and $fl(x)$ is its floating-point representation, then:*

$$fl(x) = x(1 + \delta), \quad |\delta| \leq \varepsilon_{mach}$$

2. *For basic arithmetic operations $\circ \in \{+, -, \times, \div\}$:*

$$fl(x \circ y) = (x \circ y)(1 + \delta), \quad |\delta| \leq \varepsilon_{mach}$$

3. *Gradual underflow through denormalized numbers*
4. *Consistent handling of special cases*

Intuition: Understanding floating-point precision

Floating-point numbers have non-uniform spacing throughout the real line:

- Near zero, numbers are closely spaced (spacing $\approx 2^{-149}$ for single precision)
- As magnitude increases, spacing increases proportionally (spacing near x is $\approx \varepsilon_{mach} \cdot |x|$)
- This relative precision is what makes floating-point arithmetic useful for scientific computation
- The "machine epsilon" represents the fundamental limit of relative precision

Think of floating-point numbers as having a fixed number of significant digits, regardless of magnitude.

---

2 Roundoff Error Analysis

2.1 Basic Error Bounds

Definition. Machine Epsilon

The machine epsilon ε_{mach} is the smallest positive number such that $1 + \varepsilon_{mach} > 1$ in floating-point arithmetic.

For IEEE 754:

- Single precision: $\varepsilon_{mach} = 2^{-23} \approx 1.19 \times 10^{-7}$
- Double precision: $\varepsilon_{mach} = 2^{-52} \approx 2.22 \times 10^{-16}$

Theorem 2.1 (Standard Model of Floating-Point Arithmetic)**.** *For floating-point numbers x and y and operation $\circ \in \{+, -, \times, \div\}$:*

$$fl(x \circ y) = (x \circ y)(1 + \delta), \quad |\delta| \leq \varepsilon_{mach}$$

This model assumes that the exact result of the operation is within the representable range.

Theorem 2.2 (Error Accumulation in Summation)**.** *For the floating-point summation of n numbers $s_n = \sum_{i=1}^{n} x_i$:*

$$|fl(s_n) - s_n| \leq \gamma_n \sum_{i=1}^{n} |x_i|$$

where $\gamma_n = \frac{n\varepsilon_{mach}}{1-n\varepsilon_{mach}}$ for $n\varepsilon_{mach} < 1$.

Proof sketch. By induction on n. For $n = 2$:

$$fl(x_1 + x_2) = (x_1 + x_2)(1 + \delta_1), \quad |\delta_1| \leq \varepsilon_{mach}$$

For $n > 2$, assuming the result holds for $n - 1$:

$$fl(s_n) = fl(fl(s_{n-1}) + x_n) = (fl(s_{n-1}) + x_n)(1 + \delta_n)$$

Substituting the induction hypothesis and simplifying gives the result. □

2.2 Catastrophic Cancellation

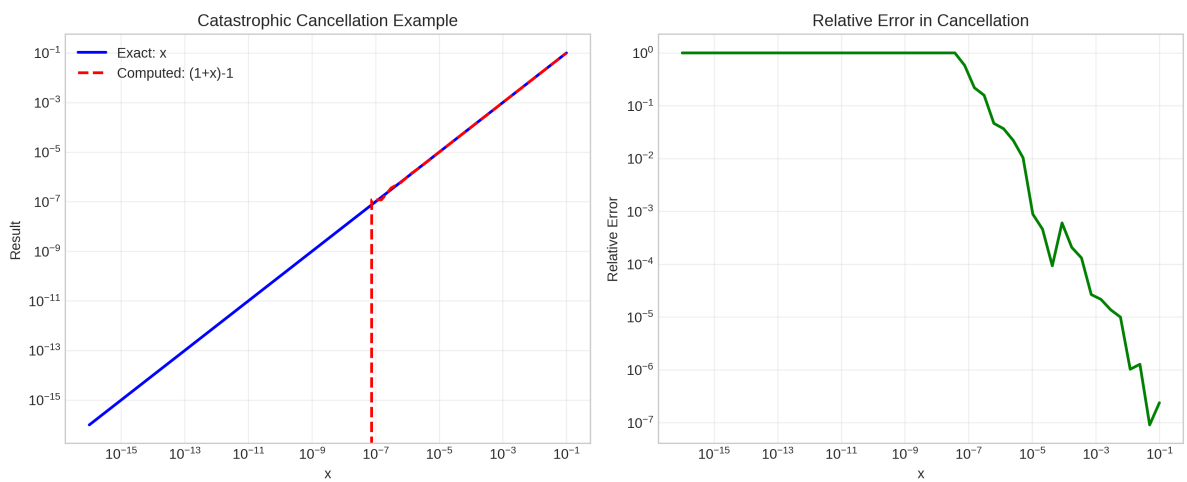

Figure 2: Catastrophic cancellation in floating-point arithmetic. The left panel shows how computing $(1 + x) - 1$ loses precision for small values of x. The right panel shows the relative error growing as x decreases, demonstrating the severity of the problem.

Definition. Catastrophic Cancellation

Catastrophic cancellation occurs when subtracting nearly equal floating-point numbers, resulting in a significant loss of precision.

If x and y have p significant digits and $|x - y| \ll |x|$, then $x - y$ may have far fewer than p significant digits.

Example 2.1 (Catastrophic Cancellation)**.** *Consider computing $\sqrt{x+1} - 1$ for small x:*

Direct computation: *For $x = 10^{-16}$ in double precision:*

$$\sqrt{1 + 10^{-16}} \approx 1.0000000000000000 \tag{1}$$

$$\sqrt{1 + 10^{-16}} - 1 \approx 0.0000000000000000 \tag{2}$$

All significant digits are lost.

Algebraic reformulation:

$$\sqrt{x+1}-1 = \frac{(\sqrt{x+1}-1)(\sqrt{x+1}+1)}{\sqrt{x+1}+1} \tag{3}$$

$$= \frac{x}{\sqrt{x+1}+1} \tag{4}$$

For $x = 10^{-16}$*:*

$$\frac{10^{-16}}{1+1} = 5 \times 10^{-17} \tag{5}$$

This preserves accuracy.

Theorem 2.3 (Cancellation Error Bound)**.** *When computing* $x - y$ *where* x *and* y *are approximate values with relative errors* ϵ_x *and* ϵ_y*:*

$$\frac{|(x-y)-(x_{exact}-y_{exact})|}{|x_{exact}-y_{exact}|} \approx \frac{|x\epsilon_x - y\epsilon_y|}{|x-y|}$$

If $x \approx y$ *and* $\epsilon_x \approx \epsilon_y \approx \epsilon$*, this becomes:*

$$\frac{|x-y|_{error}}{|x-y|} \approx \frac{|x|\epsilon}{|x-y|} \gg \epsilon$$

Intuition: Why cancellation is catastrophic

Floating-point numbers store a fixed number of significant digits, with the decimal point "floating" to represent a wide range of magnitudes.
When subtracting nearly equal numbers:

- The leading digits cancel out
- The result depends on the least significant digits
- These least significant digits contain the most roundoff error
- The relative error in the result can be magnified dramatically

This is why algorithms should be designed to avoid subtracting nearly equal quantities whenever possible.

---

3 Numerical Stability Analysis

3.1 Forward and Backward Error

Definition. Forward and Backward Error in Floating-Point

For a mathematical operation f and its floating-point implementation $fl(f)$:
Forward Error: $|f(x) - fl(f)(x)|$
Backward Error: $\min\{|\Delta x| : f(x+\Delta x) = fl(f)(x)\}$

Theorem 3.1 (Backward Error Analysis)**.** *For basic floating-point operations:*

- *Addition/subtraction:* $fl(x \pm y) = (x \pm y)(1+\delta)$ *for some* $|\delta| \leq \varepsilon_{mach}$
- *Multiplication:* $fl(x \times y) = (x \times y)(1+\delta)$ *for some* $|\delta| \leq \varepsilon_{mach}$
- *Division:* $fl(x \div y) = (x \div y)(1+\delta)$ *for some* $|\delta| \leq \varepsilon_{mach}$

These operations produce the exact result for slightly perturbed inputs.

Example 3.1 (Forward vs. Backward Error)**.** *Consider computing* $y = x^2$ *where* $x = 3.14159$ *is rounded to* $\hat{x} = 3.14$.

Forward error: $|x^2 - \hat{x}^2| = |9.86959 - 9.8596| \approx 0.01$

Backward error: *Find* Δx *such that* $(x + \Delta x)^2 = \hat{x}^2$ $\Delta x = \hat{x} - x = 3.14 - 3.14159 \approx -0.00159$

The backward error is much smaller than the forward error.

3.2 Stability of Numerical Algorithms

Definition. Numerical Stability

An algorithm is **numerically stable** if the computed solution is the exact solution to a nearby problem.

An algorithm is **backward stable** if for any input x, the computed solution $\hat{y}$ satisfies:

$$\hat{y} = f(x + \Delta x)$$

where $\|\Delta x\|/\|x\|$ is small (on the order of ε_{mach}).

Theorem 3.2 (Stability of Summation Algorithms)**.** *For summing* n *floating-point numbers:*

Naive summation *(in order* $x_1, x_2, \ldots, x_n$*):*

$$|fl(s_n) - s_n| \leq \gamma_{n-1} \sum_{i=1}^{n} |x_i|$$

Pairwise summation *(recursive divide-and-conquer):*

$$|fl(s_n) - s_n| \leq \gamma_{\log_2 n} \sum_{i=1}^{n} |x_i|$$

Pairwise summation has significantly better error bounds for large n.

Example 3.2 (Compensated Summation)**.** *Kahan's compensated summation algorithm:*

```
s = x[1]
c = 0
for i = 2 to n:
    y = x[i] - c
    t = s + y
    c = (t - s) - y
    s = t
```

This algorithm achieves:

$$|fl(s_n) - s_n| \leq 2\varepsilon_{mach} \sum_{i=1}^{n} |x_i| + O(\varepsilon_{mach}^2)$$

The error is independent of n*, making it suitable for summing large sequences.*

Intuition: Designing stable algorithms

Numerical stability is about controlling how errors propagate through computation:

- Unstable algorithms amplify roundoff errors
- Stable algorithms keep errors proportional to the inherent problem conditioning
- Backward stability is often the most practical goal
- Sometimes reformulating the mathematical approach is necessary for stability

The art of numerical analysis lies in designing algorithms that remain accurate despite the limitations of finite-precision arithmetic.

---

4 Advanced Topics in Error Analysis

4.1 Interval Arithmetic

Definition. Interval Arithmetic

Interval arithmetic represents each quantity as a range $[a, b]$ that is guaranteed to contain the exact value. Basic operations:

$$[a, b] + [c, d] = [a + c, b + d] \tag{6}$$
$$[a, b] - [c, d] = [a - d, b - c] \tag{7}$$
$$[a, b] \times [c, d] = [\min(ac, ad, bc, bd), \max(ac, ad, bc, bd)] \tag{8}$$
$$[a, b] \div [c, d] = [a, b] \times [1/d, 1/c] \quad \text{if } 0 \notin [c, d] \tag{9}$$

Theorem 4.1 (Interval Arithmetic Properties)**.** *1. Containment: If $x \in [a, b]$ and $y \in [c, d]$, then $x \circ y \in [a, b] \circ [c, d]$ for $\circ \in \{+, -, \times, \div\}$*

2. *Inclusion monotonicity: If $[a, b] \subseteq [a', b']$ and $[c, d] \subseteq [c', d']$, then $[a, b] \circ [c, d] \subseteq [a', b'] \circ [c', d']$*

3. *Interval width grows with computation: $w([a, b] \circ [c, d]) \geq \max(w([a, b]), w([c, d]))$ where $w([a, b]) = b - a$*

4.2 Probabilistic Error Analysis

Theorem 4.2 (Probabilistic Roundoff Error)**.** *Under the assumption that roundoff errors are independent random variables with zero mean:*

For the summation of n floating-point numbers with similar magnitudes, the expected error is:

$$E[|fl(s_n) - s_n|] \approx \frac{\varepsilon_{mach}}{\sqrt{3}} \sqrt{n} \sum_{i=1}^{n} |x_i|$$

This $O(\sqrt{n})$ growth is often observed in practice, compared to the worst-case $O(n)$ bound.

Example 4.1 (Monte Carlo Error Estimation)**.** *For complex algorithms where formal error analysis is intractable:*

1. *Run the algorithm with different rounding modes*
2. *Run the algorithm with perturbed inputs*
3. *Compare results from different precision levels*
4. *Use statistical methods to estimate error distributions*

This approach provides practical error estimates for complex numerical software.

Practice Problems

1. Prove that the machine epsilon for base-β floating-point with p digits is $\varepsilon_{mach} = \frac{1}{2}\beta^{1-p}$.
2. Show that the relative error in computing $\sqrt{x^2+y^2}$ can be large when $|y| \ll |x|$. Derive a more stable algorithm.
3. Analyze the error in the quadratic formula $\frac{-b\pm\sqrt{b^2-4ac}}{2a}$ when $b^2 \gg 4ac$. Provide a numerically stable implementation.

Solution:

Problem 1: In a base-β system with p digits, the smallest positive normalized number is $1.0...0 \times \beta^{-e_{max}}$. The gap between 1 and the next representable number is β^{-p+1}. Since machine epsilon is defined as the smallest ε such that $1+\varepsilon > 1$ in floating-point arithmetic, we have $\varepsilon_{mach} = \frac{1}{2}\beta^{1-p}$ (the factor $\frac{1}{2}$ appears because of rounding to nearest).

Problem 2: When computing $\sqrt{x^2+y^2}$ with $|y| \ll |x|$: $x^2+y^2 \approx x^2$ due to limited precision, leading to $\sqrt{x^2+y^2} \approx |x|$.

A more stable algorithm:

$$\sqrt{x^2+y^2} = |x|\sqrt{1+(y/x)^2} \tag{10}$$

$$= \begin{cases} |x|\sqrt{1+(y/x)^2} & \text{if } |x| \geq |y| \\ |y|\sqrt{1+(x/y)^2} & \text{if } |y| > |x| \end{cases} \tag{11}$$

This ensures we're always computing $\sqrt{1+z^2}$ where $|z| \leq 1$, avoiding loss of significance.

Problem 3: When $b^2 \gg 4ac$, computing $\sqrt{b^2-4ac}$ is approximately $|b|$, leading to catastrophic cancellation in $-b+\sqrt{b^2-4ac}$ when $b > 0$.

Stable implementation:

$$x_1 = \frac{-b-\text{sign}(b)\sqrt{b^2-4ac}}{2a} \tag{12}$$

$$x_2 = \frac{c}{ax_1} \tag{13}$$

This uses the fact that $x_1 \cdot x_2 = \frac{c}{a}$ for the roots of $ax^2+bx+c=0$, avoiding the problematic subtraction.

Summary and Looking Ahead

This week has established the mathematical foundations of computer arithmetic and error analysis. We have explored the IEEE 754 floating-point standard, analyzed how roundoff errors arise and propagate, and developed strategies for designing numerically stable algorithms. The key insights include:

- Floating-point arithmetic provides a systematic approximation of real numbers with inherent limitations
- Roundoff errors are inevitable but can be analyzed and controlled through careful algorithm design
- Catastrophic cancellation represents a fundamental challenge requiring mathematical reformulation
- Backward error analysis provides a powerful framework for understanding numerical stability
- Advanced techniques like compensated summation and interval arithmetic offer practical tools for managing roundoff error

Next week, we will apply these principles to the fundamental problem of finding roots of nonlinear equations. We will analyze bisection, fixed-point iteration, Newton's method, and secant methods, examining their convergence properties and numerical stability. The error analysis techniques developed this week will provide essential tools for understanding the practical performance of these root-finding algorithms.